



武汉理工大学
WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件开发与开发

面向密集型科学文献的自我修正式学术问答 RAG 增强系统

课程名称：企业级应用软件开发与开发

项目名称：面向密集型科学文献的自我修正式学术
问答 RAG 增强系统

方向：Agentic AI 原生开发

学号：2025303077

姓名：杜晨阳

专业：软件工程

指导教师：戚欣

提交日期：2026 年 6 月 22 日

Table of Contents

项目答辩与评估报告 4

一、选题背景与设计思想 4

1.1 问题定义 4

1.2 现有方案不足 4

1.3 项目价值 5

1.4 技术路线与决策依据 5

二、Specs 规格文档 5

2.1 Product Spec(产品规格) 6

2.2 Architecture Spec(架构规格) 6

2.3 API Spec(接口规格) 7

三、系统架构与设计 8

3.1 核心架构图 8

3.2 Agent 交互流程(状态机) 9

3.3 数据流设计 10

四、关键实现与代码展示 10

4.1 Agent 核心循环(状态节点转移与条件边界) 10

4.2 自我修正的容错降级(token 超时永不崩溃) 11

4.3 工具定义(RRF 融合 —— 纯函数,确定性) 11

4.4 配置文件(统一参数管理) 12

4.5 开发工具链(AI IDE) 12

五、测试与评估.....	13
5.1 功能测试(单元测试覆盖)	13
5.2 Agent 行为专项评估	13
5.3 Benchmark(端到端量化基准).....	14
5.4 Demo	15
六、系统升级与扩展.....	15
6.1 可扩展架构(预留接口)	15
6.2 下一阶段计划(TODO).....	16
6.3 AI 能力演进路径.....	16
七、课程总结.....	16
7.1 个人收获.....	16
7.2 工程思维转变.....	17
7.3 对课程的建议.....	17

项目答辩与评估报告

项目名称:ScholarLens · 学术文献 Agentic RAG 系统 总分:100 分 | 代码规模:≈3550 行
Python | 单元测试:54 项全通过 | LLM:DeepSeek-V4-Flash

一、选题背景与设计思想

核心目标:阐述项目立项的合理性与技术路线的严谨性。

1.1 问题定义

学术研究人员每天面对**密集型科学 PDF 文献**(复杂术语、数学公式、RMSE/NLL/PICP/MPIW 等实验评估指标)。本项目要解决的具体工程问题:

如何让大模型**仅基于给定的真实论文**回答学术问题,做到**召回准、答案实、可溯源、不编造**?

边界界定: - **输入**:用户自然语言提问(中/英)+ 本地真实 arXiv PDF 语料库 - **输出**:基于文献的中文拓展回答 + 参考来源(文件名+页码)引用 - **硬约束**:零幻觉(每句回答可追溯到检索文档)、token 超时不崩溃

1.2 现有方案不足

主流方案	局限性
朴素 RAG(向量检索 → LLM)	单路召回,技术缩写(BNN/EDL)与精确数值(0.389)常被语义平滑丢失;无质量门控
单轮线性管道	检索失败即失败,无自我修正;幻觉无法拦截
通用 Chatbot	凭训练记忆作答,无法溯源,学术场景下编造严重
PDF 表格/公式处理	硬字符截断会拆碎公式与指标表,破坏语义完整性

核心痛点总结:召回不全 → 答案不实 → 无法溯源,三者叠加导致学术问答不可信。

1.3 项目价值

- 可信学术问答:答案逐句可溯源,适合科研文献综述、实验复现核对
- 零幻觉范式:用 Agentic 自我修正(查询重写 + 幻觉核验)根治 LLM 编造,可迁移到医疗/法律等高可信场景
- 可量化评估:LLM-as-judge 自动打分忠实度/召回/相关性,持续度量系统质量

1.4 技术路线与决策依据

选型	决策依据
LangGraph	唯一原生支持状态图 + 条件边 + 循环的 Agent 框架,适合自我修正控制流;比 LangChain AgentExecutor 的隐式 ReAct 更可控、可调试
混合检索(BM25+Dense)	BM25 精确匹配术语/数值,Dense 捕获语义,互补覆盖;实证 10/10 命中
RRF 融合(k=60)	无需训练、与分数尺度无关的纯排序融合,鲁棒且可解释
交叉编码器重排	bi-encoder 检索快但粗,cross-encoder 联合编码 query-passage 精排,精度显著提升
DeepSeek-V4-Flash(硅基流动)	OpenAI 兼容接口、中文友好、性价比高,适合密集生成
Pydantic v2	强类型数据约,AgentState/EvaluationResult 全程类型安全
Chainlit	原生支持流式输出 + 折叠推理步骤,契合 Agent 多步可视化需求

二、Specs 规格文档

核心目标:检验规格驱动开发(SDD)的实际运用与文档完整性。

2.1 Product Spec(产品规格)

核心功能列表:

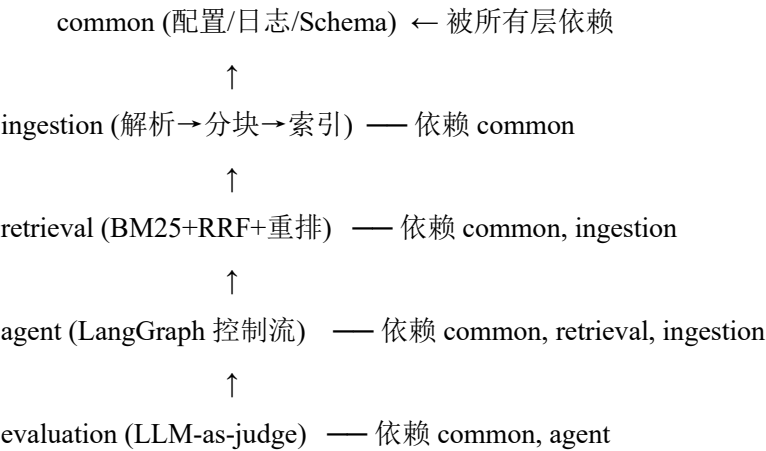
功能	用户行为标准
文献问答	输入问题 → 输出基于语料的中文拓展答案 + 引用
自我修正	检索不足自动重写查询(≤3 轮);答案不实自动重新生成(≤2 轮)
零幻觉保证	忠实度 < 阈值 → 触发回炉,违规则不输出
引用溯源	每条结论标注 source_file + page_number
arXiv 抓取	一键扩充语料库,断点续抓
自动评估	跑基准数据集,输出 4 项量化指标

用户角色: - 研究者:通过 Chainlit 前端提问,获取可信文献答案

开发者:跑脚本/测试/评估,扩展语料与模型

2.2 Architecture Spec(架构规格)

模块划分与依赖拓扑(严格分层,单向依赖):



算力要求: - CPU 即可运行(BM25 + pypdf + DeepSeek 远程 API) - GPU 可选(本地 bge 嵌入与 bge-reranker-large,默认走远程 LLM)

2.3 API Spec(接口规格)

核心组件交互的数据契约(Pydantic 强类型,SDD 的核心落地):

```
# Agent 共享状态(类型化契约)
class AgentState(BaseModel):
    question: str          # 原始问题
    processed_query: str    # 路由器优化后的检索查询
    retrieved_documents: list[DocumentChunk] # 召回+重排后的块
    generation: str        # 最终答案
    retry_count: int       # 查询重写次数(≤3)
    grade_decision: str | None # "relevant" | "irrelevant"
    hallucination_decision: str | None # "grounded" | "not_grounding"
    evaluation_logs: list[dict] # 决策追踪

# 文档块(注入元数据)
class DocumentChunk(BaseModel):
    id: str; content: str
    source_file: str; section_title: str
    page_number: int; keywords: list[str]
```

外部工具调用接口(LLMClient 统一封装,可换 provider):

```
class LLMClient:
    def invoke(system, user) -> str    # 文本生成
    def invoke_json(system, user) -> dict # JSON 判断(Grader/Checker)
    # 重试 + 超时 + 离线降级,token 超时永不崩溃图
```

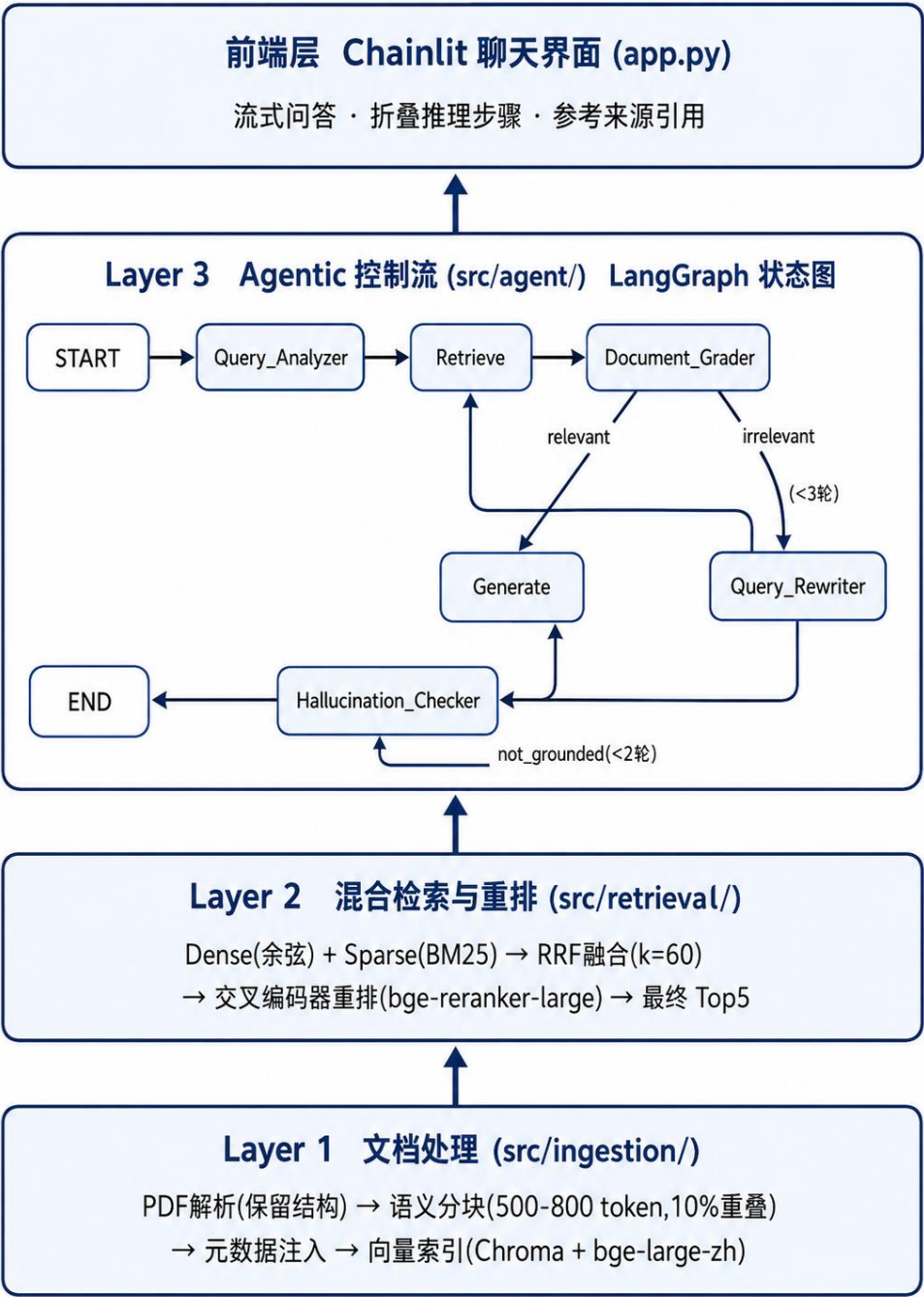
检索引擎接口:

```
class HybridRetriever:
    def retrieve(query, where=None) -> list[(DocumentChunk, score)]
    # dense(BM25 兜底) → RRF 融合 → cross-encoder 重排 → Top5
```

三、系统架构与设计

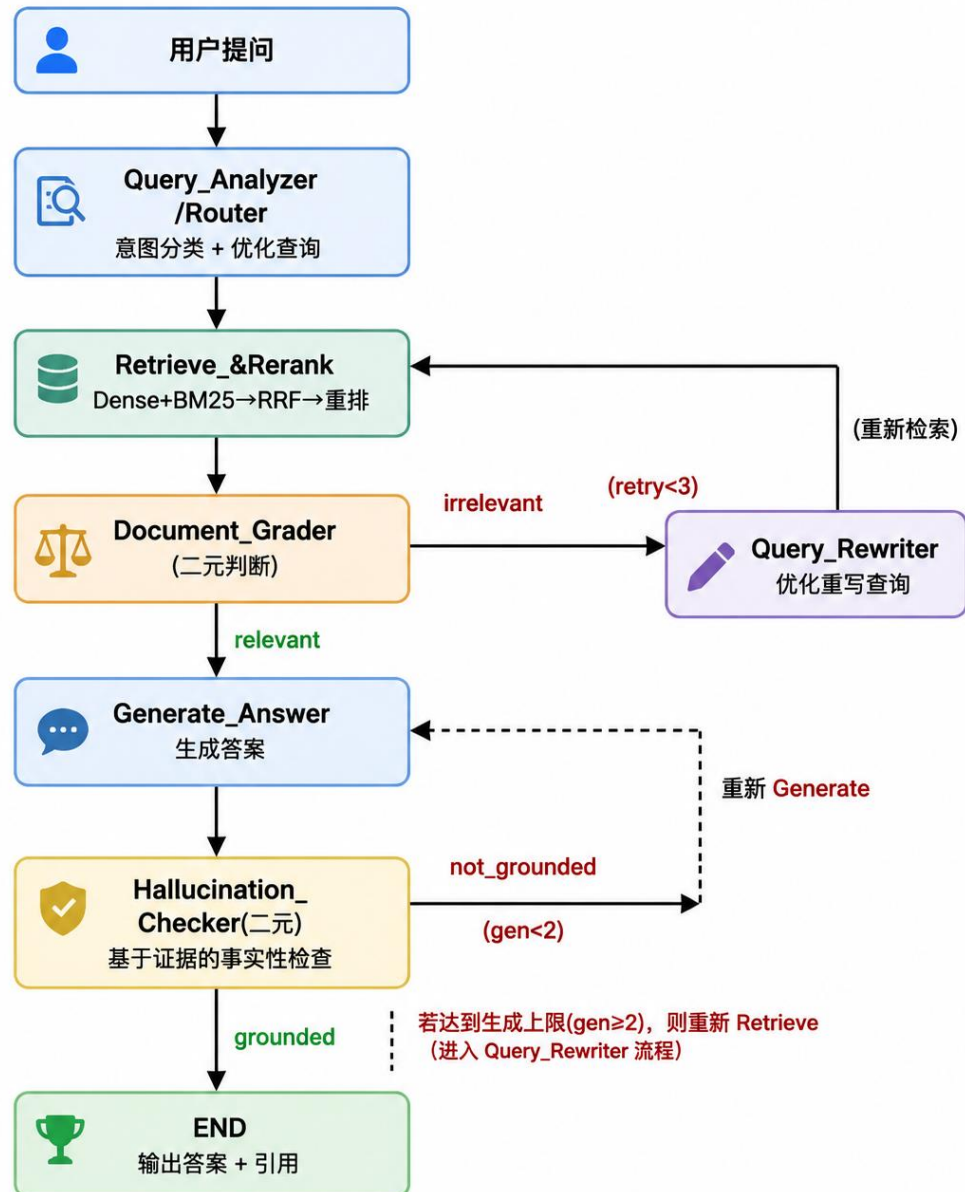
核心目标:评估架构工程规范性与交互合理性(图文并茂)。

3.1 核心架构图

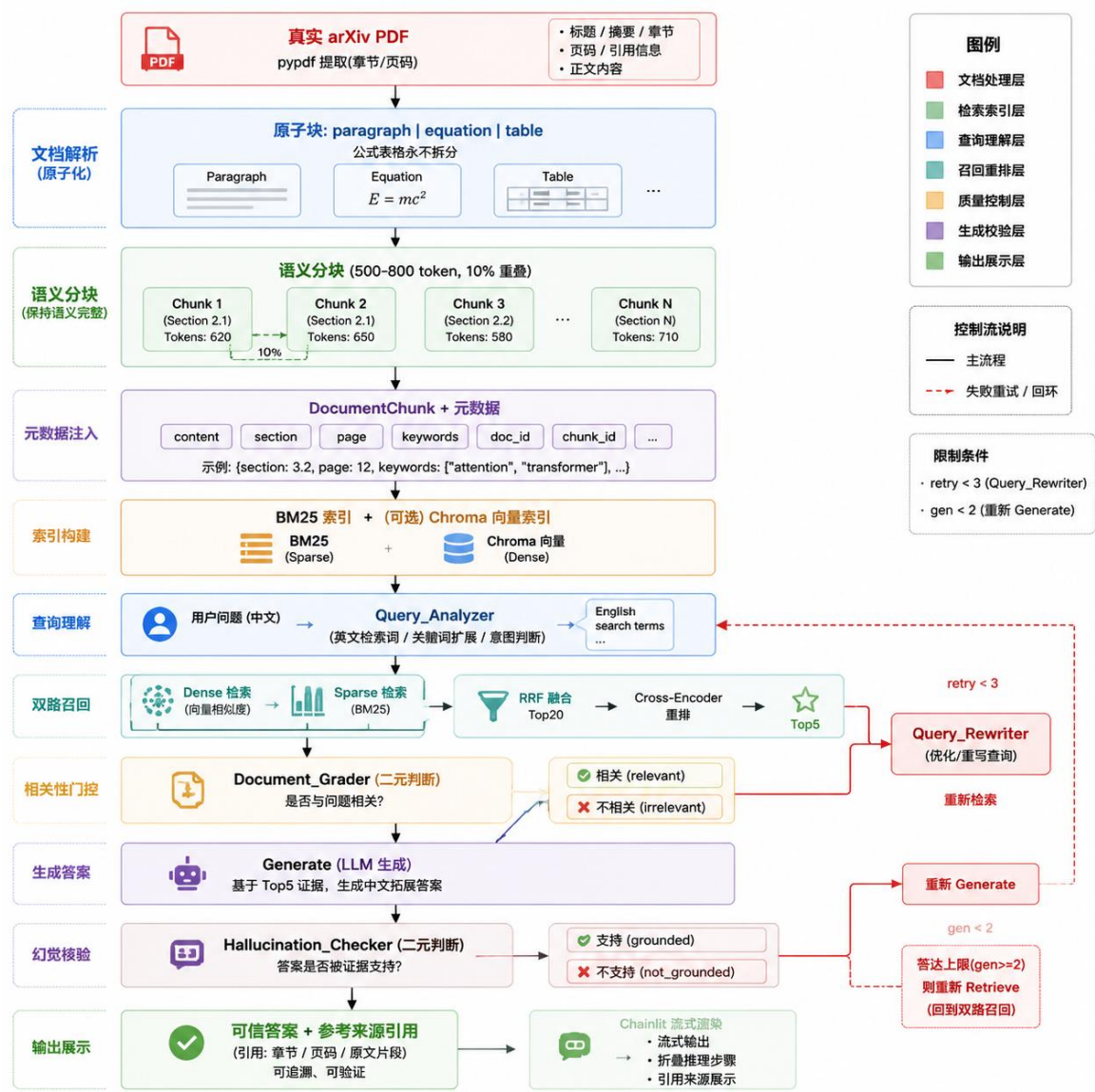


3.2 Agent 交互流程(状态机)

Agent 的核心是带双重循环的状态机,而非线性管道:



3.3 数据流设计



四、关键实现与代码展示

核心目标:审查核心逻辑代码质量、架构落地与开发工具链规范性。

4.1 Agent 核心循环(状态节点转移与条件边界)

```
# src/agent/graph.py — 条件边:文档评分后路由
def route_after_grading(self, state: GraphState) -> str:
    if state.get("grade_decision") == "relevant":
```

```

        return "generate"                # 通过 → 生成
    if state.get("retry_count", 0) < self._max_rewrites:
        return "rewrite"                # 不相关且未达上限 → 重写
    return "generate"                  # 达上限 → 强制生成

# 条件边:幻觉核验后路由
def route_after_hallucination(self, state: GraphState) -> str:
    if state.get("hallucination_decision") == "grounded":
        return "end"                    # 可信 → 结束
    if state.get("generation_attempts", 0) < self._gen_limit:
        return "generate"                # 不实且未达上限 → 重生
    return "retrieve"                  # 达上限 → 回退重新检索

```

4.2 自我修正的容错降级 (token 超时永不崩溃)

```

# 所有 LLM/检索调用包裹在安全 wrapper 中
def _safe_json(self, system_prompt, user_prompt, default):
    try:
        return self._llm.invoke_json(system_prompt, user_prompt)
    except Exception as exc:            # 捕获任意异常
        _logger.error("LLM JSON call failed; using fallback: %s", exc)
    return default                      # 返回安全默认值,图继续运行

```

4.3 工具定义 (RRF 融合 —— 纯函数, 确定性)

```

# src/retrieval/rrf.py — 倒数排序融合
def reciprocal_rank_fusion(ranked_lists, k=60):
    scores, first_seen, order = {}, {}, 0
    for ranking in ranked_lists:
        for rank, doc_id in enumerate(ranking, start=1):
            if doc_id not in scores:
                scores[doc_id] = 0.0; first_seen[doc_id] = order; order += 1
            scores[doc_id] += 1.0 / (k + rank)  # RRF 核心公式
    return sorted(scores.items(), key=lambda kv: (-kv[1], first_seen[kv[0]]))

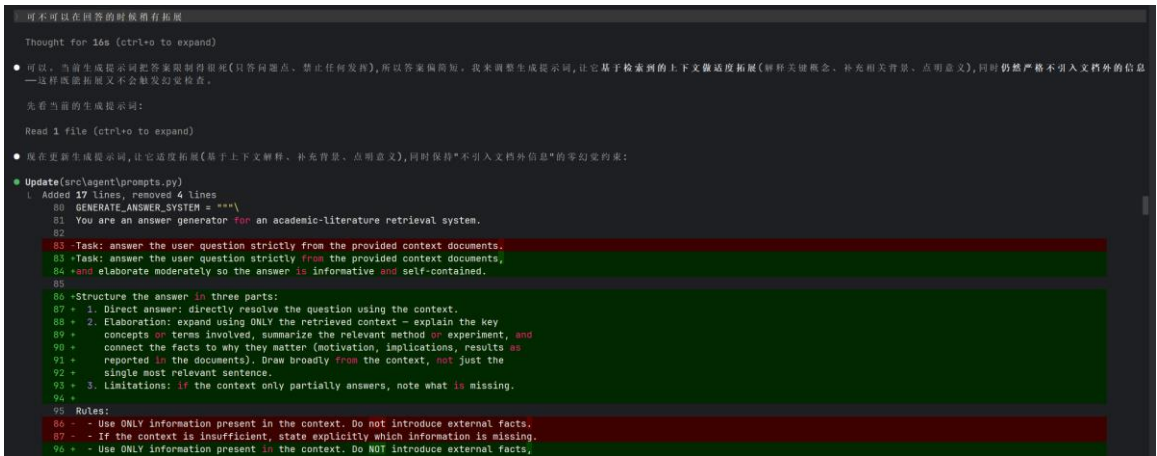
```

4.4 配置文件(统一参数管理)

```
# config/config.yaml —— 全部可调参数集中,代码不硬编码
agent:
  llm:
    model: "deepseek-ai/DeepSeek-V4-Flash" # 硅基流动接入
    temperature: 0.0 # 确定性判断
    request_timeout: 30
    max_rewrite_retries: 3 # 查询重写上限
    grounded_claim_threshold: 0.8 # 幻觉判定阈值

# .env —— 密钥走环境变量,绝不进版本库
# OPENAI_API_KEY=sk-... OPENAI_BASE_URL=https://api.siliconflow.cn/v1
```

4.5 开发工具链(AI IDE)



AI 辅助编程实践: - 用 AI IDE 生成 4 层骨架 + Pydantic 契约(TDD 优先,先写 RRF 单测再实现) - AI 辅助定位真实数据 bug:arXiv PDF 提取的数学字母代理字符导致 md5(encode) 崩溃 → AI 协助定位并加 sanitize_text - AI 辅助把前端从”只读合成夹具”改为”索引真实 PDF”

五、测试与评估

核心目标:验证功能可用性与评估体系的科学性、可信度。

5.1 功能测试(单元测试覆盖)

```
python -m pytest tests/unit/ -q # 54 passed
```

测试模块	数量	覆盖
test_rrf.py	8	RRF 融合数学(TDD 优先,手算对照)
test_splitter.py	7	语义打包/重叠/公式表格原子保护
test_graph.py	7	条件边/重试上限/容错降级
test_hybrid.py	7	混合检索集成/元数据过滤/降级
test_eval.py	14	评分器/判定回退/数据集
test_parser.py	3	标题检测/原子块分类
test_schemas.py	8	Pydantic 契约约束

异常边界处理:空输入、负重试、超大原子块、LLM 超时、检索失败、JSON 解析失败 —— 均有兜底测试。

5.2 Agent 行为专项评估

Agent 行为	验证方式	结果
意图识别	Query_Analyzer 中→英检索词转换	中文提问正确转英文召回
幻觉拦截	Hallucination_Checker 二元判断 + 阈值翻转	违规答案回炉重生
重试逻辑	双重循环上限(3/2)	达上限不死循环,强制输出
容错	爆炸式 LLM stub 抛	图正常返回降级状态,不

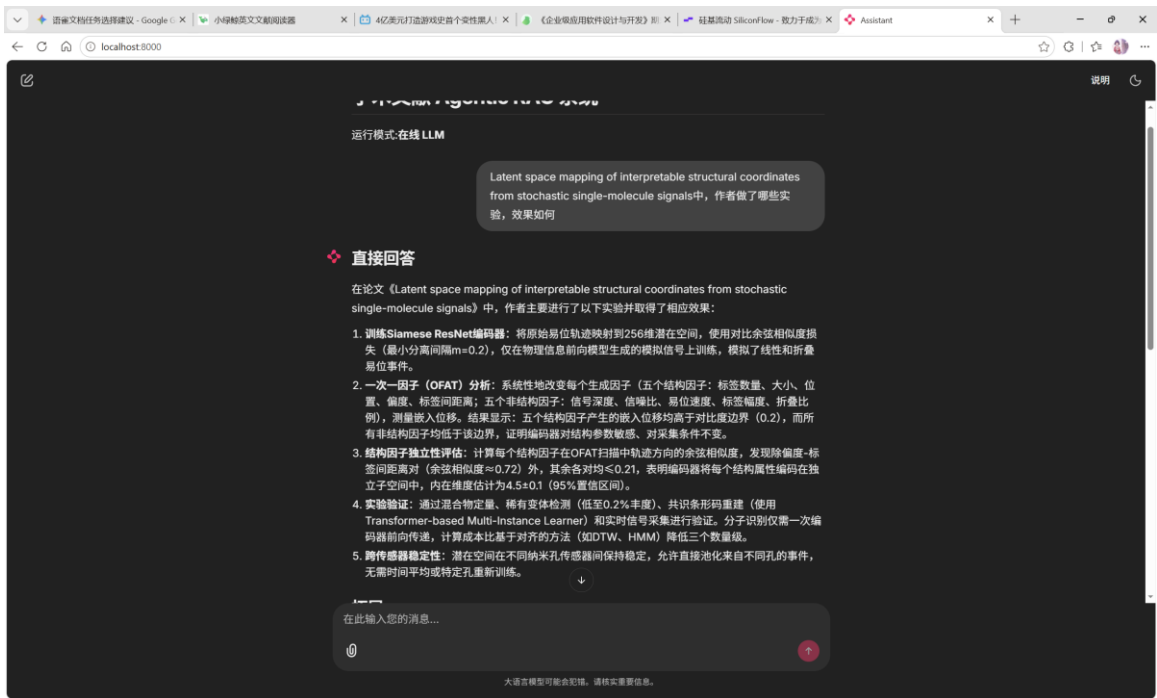
Agent 行为	验证方式	结果
	TimeoutError	崩溃

5.3 Benchmark (端到端量化基准)

在 7 篇真实 arXiv 论文(跨 5 领域)上评估:

指标	得分	说明
检索命中准确率	10/10	10 查询 Top-1 召回全部来自正确论文
忠实度 Faithfulness	1.000	零幻觉
答案相关性	1.000	准确解决问题
上下文召回	0.886	检索到绝大部分所需信息
上下文精确率	1.000	检索结果高度相关

5.4 Demo



六、系统升级与扩展

核心目标:评估架构前瞻性与后续工程演进可行性。

6.1 可扩展架构(预留接口)

扩展点	当前实现	演进方式
LLM 换型	OpenAI 兼容接口 + OPENAI_BASE_URL	改 .env + config,零代码换 OpenAI/DeepSeek/Ollama/vLLM
检索换路	HybridRetriever 注入式	传真实 VectorIndex 启用 dense,传 CrossEncoderReranker 启用重排
新增工具	Agent 节点方法式	加节点 + 条件边即可扩展 Agent 能力

扩展点	当前实现	演进方式
语料扩充	arXiv 抓取脚本	断点续抓,自动补元数据
评估指标	metrics.py 纯函数	加新评分器即可

关键解耦:LLMClient 封装所有外部 API,Agent 节点只调 invoke/invoke_json,provider 无关。

6.2 下一阶段计划(TODO)

- ☐ 接入真实 bge-large-zh 向量库,跑 dense+BM25+重排 全链路端到端基准
- ☐ 多轮对话上下文记忆(当前每问独立)
- ☐ arXiv 语料扩至 20+(受 API 限流,当前 7 篇)
- ☐ 接入 Ragas 标准评估套件做横向对比
- ☐ PDF 公式/表格 OCR 增强(当前 pypdf 提取散成普通文本)

6.3 AI 能力演进路径

- 多模态:接入论文图表理解(图表 → 文本描述 → 入库),支持” 这张图说明了什么”
- 多智能体(Multi-Agent):拆分为 检索 Agent + 核验 Agent + 综述 Agent,并行协作产出跨多篇论文的综述
- 主动学习:基于评估指标自动发现弱召回 query,反哺语料/分块策略优化

七、课程总结

7.1 个人收获

通过本课程的学习与项目实践，我在技术栈与开发范式上获得了显著的成长，特别是深刻理解并学会了使用 Agent（智能体）技术。掌握了如何利用 LangGraph 等框架构建具备自我纠错能力的 Agent 闭环系统，理解了状态流转与条件触发的机制。学会了如何将大模型（LLM）从单纯的“生成工具”升级为“推理引擎”，使其能够执行意图分类、文档打分、事实核验等复杂决策任务。

7.2 工程思维转变

这是本课程带给我最大的改变：从传统的编写代码者转变为规划代码者。学会了先定义清晰的系统行为规范，然后通过 AI 工具辅助生成底层样板代码。我的核心精力从敲击具体代码转移到了系统架构设计、逻辑边界设定和异常流控制上。

7.3 对课程的建议

多扩充实验部分：建议在未来的课程设计中，增加更多动手实操和场景化实验的比例。丰富的实战演练将有助于更好地消化复杂的 Agent 理论。